3. Banker’s algorithm

```
maitreyee@localhost:~
[maitreyee@localhost ~]$ ls
-       bank      Desktop    mait     Music     sample.txt  while.sh
a       banker.c  Documents  maithu   Pictures  sh          whilw.sh
ABC     bankers   Downloads  maitre   Public    Templates   XYZ
abc.sh  bm        h          mash     REC       test.sh
ab.sh   close     mai        mithii   sample.c  Videos
[maitreyee@localhost ~]$ vi os_lab
[maitreyee@localhost ~]$ vi os_lab.c
[maitreyee@localhost ~]$ gcc os_lab.c -o os_lab
[maitreyee@localhost ~]$ ./os_lab
```

```
[maitreyee@localhost ~]$ ./os_lab
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
Safe sequence is: P1 P3 P4 P0 P2
[maitreyee@localhost ~]$
```

Code execution :

```c
#include <stdio.h>

int main() {
    int n, r, i, j, k;

    // Input number of processes and resources
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int alloc[n][r], max[n][r], need[n][r], avail[r];
    int finish[n], safe_sequence[n];
    int count = 0, safeFound;

    // Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < r; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Max Matrix
    printf("Enter Max Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < r; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Available Resources
    printf("Enter Available Resources:\n");
    for(i = 0; i < r; i++) {
        scanf("%d", &avail[i]);
    }
```

```
                int safe = 1;

                for(j = 0; j < r; j++) {
                    if(need[i][j] > avail[j]) {
                        safe = 0;
                        break;
                    }
                }

                if(safe) {
                    for(k = 0; k < r; k++) {
                        avail[k] += alloc[i][k];
                    }

                    safe_sequence[count] = i;
                    finish[i] = 1;
                    safeFound = 1;
                    count++;
                }
            }
        }

        if(!safeFound) {
            printf("System is in unsafe state! May occur deadlock.\n");
            return 0;
        }
    }

    // Print Safe Sequence
    printf("Safe sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d ", safe_sequence[i]);
    }
    printf("\n");

    return 0;
}
```